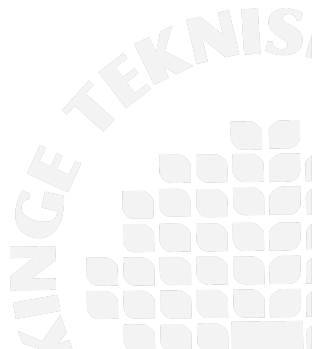


DV1655/6 - Compiler Design and Translation Technology

Semantic Analysis

Suejb Memeti

LP3 2024



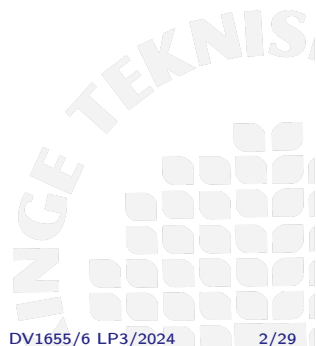
Outline

Overview

Tree Traversal

Symbol table

Semantic Analysis



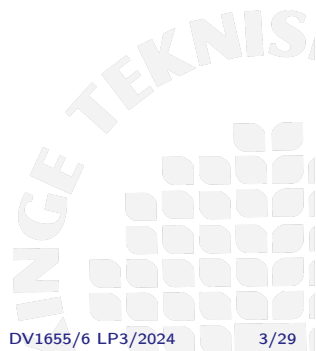
Outline

Overview

Tree Traversal

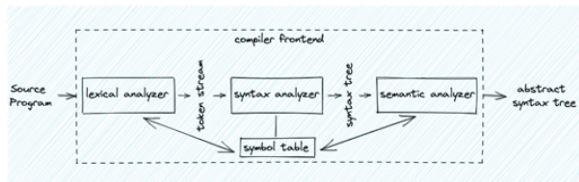
Symbol table

Semantic Analysis

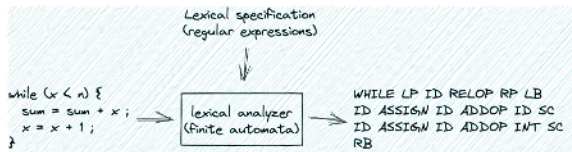


Front-end Overview

- ▶ Lexical analysis
 - ▶ Identify atomic language constructs
 - E.g. `if` $\Rightarrow$ `IF`; `2.0` $\Rightarrow$ `FLOAT`; `x` $\Rightarrow$ `ID`; ...
- ▶ Syntax analysis
 - ▶ Check if the token stream is correct with respect to language specification
- ▶ Semantic analysis
 - ▶ Check type relations and consistency rules
 - E.g. if `type(lhs) = type(rhs)` in an assignment `lhs=rhs`; ...
- ▶ Symbol table
 - ▶ Contains information about the source program. Used by all phases of the compiler.



Lexical Analysis Overview



► Input: program representations (character sequence)

► Output: program representation (token sequence)

► A token comprises:

Name e.g., **keywords**: IF, WHILE, CLASS, ...; , **operators**: +, -, *, <, ...;
and **literals**: STRING, INT, ID, ... ,

Value - a.k.a., **lexeme**, e.g., 12, "string" , varName,

Location - line, column, ...

► Lexical specification: regular expressions

► Lexical analysis implementation: finite automata

► Pattern

E.g. "for", [0-9]+, [a-z]+, ...

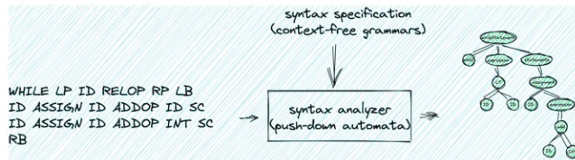
► Lexeme

E.g. for, 153, varname, ...

► Token

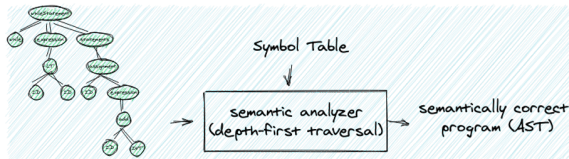
E.g. FOR, INT, IDENTIFIER, ...

Syntax Analysis Overview



- ▶ Input: program representation (token sequence)
- ▶ Output: parse (syntax) tree
- ▶ A parse tree comprises nodes and non-labeled edges.
- ▶ Syntax specification: context free grammar
- ▶ Syntax analysis implementation: push-down automata
 - ▶ Top-down and bottom-up parsers

Semantic Analysis (Type Checking) Overview



- ▶ Input: abstract syntax tree
- ▶ Output: semantically correct program
- ▶ Symbol table: constructed by visiting the AST
- ▶ Semantic analysis: performed by traversing the AST and using the information in the symbol table

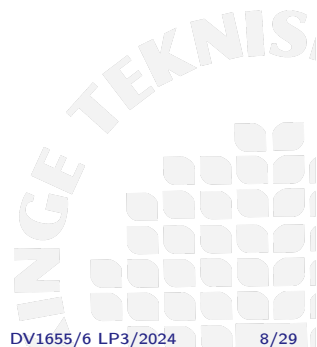
Outline

Overview

Tree Traversal

Symbol table

Semantic Analysis

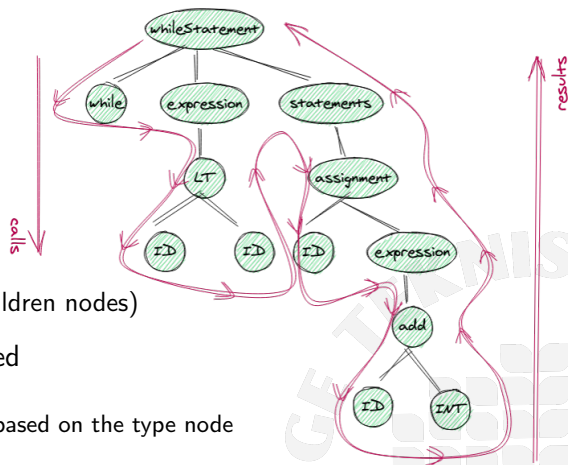


Tree Traversal

- ▶ Tree traversal
 - ▶ Recursive in nature
 - ▶ Pre-order
 - ▶ Post-order

E.g. the `print_tree()` method

- ▶ Pre-order (starts from root visit all children nodes)
- ▶ Depending on how the AST is constructed
 - ▶ Base design
 - ▶ Determine what actions to perform based on the type node
 - ▶ OO-design
 - ▶ A base class `Node` is derived by specific classes that represent specific language constructs. Each derived class implements its own functionality.



Tree traversal algorithms (overview)

► Depth-first traversal

- **In-order** (Left, Root, Right)

E.g. 4 2 5 1 3

- **Pre-order** (Root, Left, Right)

E.g. 1 2 4 5 3

- Used to construct the symbol table

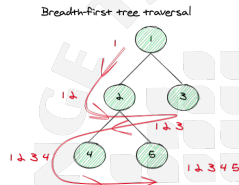
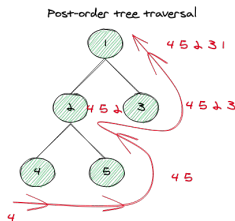
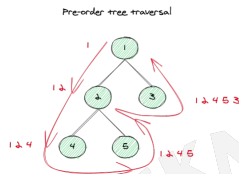
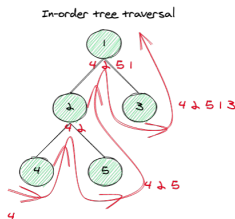
- **Post-order** (Left, Right, Root)

E.g. 4 5 2 3 1

- Used to perform the semantic analysis (type checking)

► Breadth-first traversal

E.g. 1 2 3 4 5



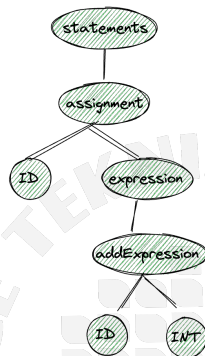
Tree traversal - base design

- ▶ A single class
- ▶ The execute method is called recursively
- ▶ Depending on what type the Node is
 - ▶ Perform specific actions

```

1  execute(...) {
2    ...
3    if(type == "statements")
4      visit all statements
5    else if(type == "assignment") {
6      lhs = execute(child[0]);
7      rhs = execute(child[1]);
8    }
9    ...
10   else if(type == "addExpression") {
11     lhs = execute(child[0]);
12     rhs = execute(child[1]);
13     result = lhs + rhs;
14   }
15   ...
16 }

```



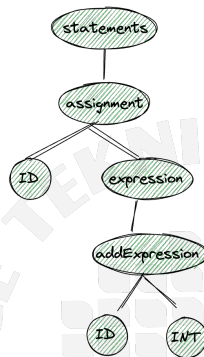
Tree traversal - OO design

- ▶ A base class
- ▶ The execute method is called recursively
- ▶ Each derived class has its own implementation of the execute method

```

1 Node() {
2   ...
3   execute(...) {...}
4   ...
5 }
6 Statements :: Node() {
7   ...
8   execute(...) {
9     visit all statements
10  }
11  ...
12 }
13 Assignment :: Node() {
14   ...
15   execute(...) {
16     lhs = execute(child[0]);
17     rhs = execute(child[1]);
18     ...
19   }
20   ...
21 }
22 AddExpression :: Node() {
23   ...
24   execute(...) {
25     lhs = execute(child[0]);
26     rhs = execute(child[1]);
27     result = lhs + rhs;
28   }
29   ...
30 }

```



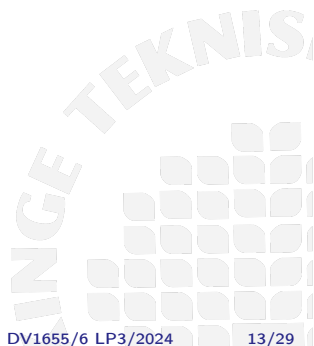
Outline

Overview

Tree Traversal

Symbol table

Semantic Analysis

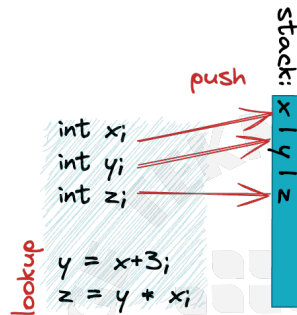


Symbol Table

- ▶ Data structures to hold information about the program constructs
E.g. variable/method/object/class... name, type, and scope
- ▶ Can be constructed during lexical analysis and syntax analysis; or after
- ▶ Used during semantic analysis, code generation, optimization, error detection
- ▶ Can be constructed in one (pre-order) traversal of the AST
- ▶ Each identifier is represented by a record where information is stored
- ▶ Features: lookup, insert, modify, delete
- ▶ Symbol tables need to support multiple declarations of the same identifier within a program
- ▶ Common implementations: trees, linked lists, hash tables

A simple symbol table

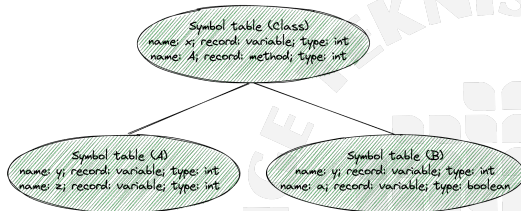
- ▶ Implemented as a stack
- ▶ Supports variable declarations
 - ▶ Symbols are added one at a time
- ▶ Operations
 - ▶ `add_symbol(x)` - push `x` to stack
 - ▶ `find_symbol(x)` - search stack for `x`.
return null if not found.
 - ▶ `remove_symbol(x)` - pop the stack
- ▶ Does not support
 - ▶ Method declarations, method parameters,
 - ▶ Variable declarations within methods



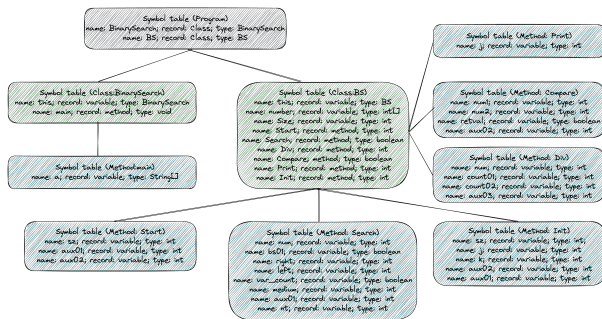
A more advanced symbol table

- ▶ Implemented as a tree
- ▶ Supports variable declarations, method, parameters, ...
- ▶ Operations
 - ▶ `enter_scope()` - start/push a new nested scope
 - ▶ `exit_scope()` - exists/pops the current scope
 - ▶ `add_symbol(x)` - push x to stack
 - ▶ `find_symbol(x)` - search stack for x. return null if not found.
 - ▶ `check_scope(x)` - returns true if x is defined in current scope
 - ▶ `remove_symbol(x)` - pop the stack
- ▶ Function names can be used before their definitions?
 - ▶ Pass 1: construct the symbol table
 - ▶ Pass 2: perform the semantic analysis
 - ▶ Semantic analysis may require more than one pass

```
int x;  
int A() {  
    int y;  
    int z;  
}  
int B() {  
    int y;  
    boolean true;  
}
```



Symbol Table: example



```

class BinarySearch {
    public static void main(String[] a) {
        System.out.println(new BS().Start(20));
    }
}

class BS {
    int[] number;
    int size;

    public int Start(int sz) {
        int aux01;
        int aux02;
        aux01 = this.Init(sz);
        aux02 = this.Print();
        if (this.Search(8)) System.out.println(1);
        else System.out.println(0);
        ...
        return 999;
    }

    public boolean Search(int num) {
        boolean bs01;
        int right;
        int left;
        boolean var_count;
        int medium;
        int aux01;
        int nt;

        aux01 = 0;
        bs01 = false;
        ...
        return bs01;
    }
}

```

```

public int Div(int num) {
    int count01;
    int count02;
    int aux03;

    count01 = 0;
    ...
    return count01;
}

public boolean Compare(int num1, int num2) {
    boolean retval;
    int aux02;

    retval = false;
    ...
    return retval;
}

public int Print() {
    int j;

    j = 1;
    while (j < (size)) {
        System.out.println(number[j]);
        ...
        return 0;
    }
}

public int Init(int sz) {
    int j;
    int k;
    int aux02;
    int aux01;

    size = sz;
    ...
    return 0;
}

```

Tree-based symbol table implementation

```

class Record {
    string id;
    string type;
    //getters and setters
    printRecord();
}

class Class: Record {
    map<string, Variable> variables;
    map<string, Method> methods;

    addVariable() { ... }
    addMethod() { ... }
    lookupVariable() { ... }
    lookupMethod() { ... }
    ...
}

class Method: Record {
    vector<Variable> parameters;
    map<string, Variable> variables;

    addVariable() { ... }
    addParameter() { ... }
    ...
}

class Variable: Record {
    ...
}

```

```

class SymbolTable {
    Scope root;
    Scope current;

    SymbolTable() { root = new Scope(null); current = root; }
    enterScope() { current = current.nextChild(); //create new scope if needed }
    exitScope() { current = current.Parent(); }

    put(String key, Record item)
    Record lookup(String key) { return current.lookup(key); }

    printTable() { root.printScope(); }
    resetTable() { root.resetScope(); //preparation for new traversal }
}

class Scope {
    int next = 0; //next child to visit
    Scope parentScope; //parent scope
    list childrenScopes; //children scopes
    map records; //symbol to record map
    ...

    Scope nextChild() {
        Scope nextChild;
        if (next == childrenScopes.size()) { //create new child Scope
            nextChild = new Scope(this);
            childrenScopes.add(nextChild);
        } else { nextChild = childrenScopes.get(next); } //visit scope
        next++;
        return nextChild;
    }
    ...
}

```

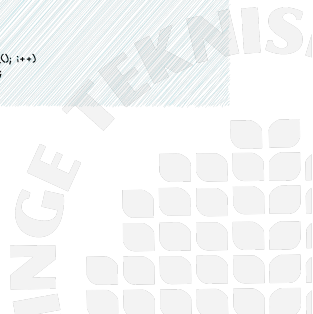
```

class Scope {
    ...

    Record lookup(string key) {
        if (records.containsKey(key)) // does it exist in the current scope?
            return record.get(key);
        else {
            if (parent == null)
                return null; //identifier not in the symbol table
            else
                return parent.lookup(key); // delegate the request to parent scope
        }
    }

    resetScope() {
        next = 0;
        for (int i = 0; i < children.size(); i++)
            children.get(i).resetScope();
    }
}

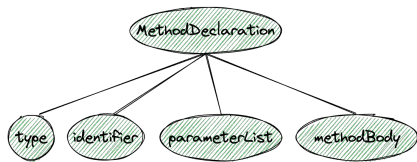
```



Summary

- ▶ Initialize the symbol table object
- ▶ Traverse the tree left-to-right
- ▶ For each detected scope in the tree
 - ▶ Create a new scope node in the symbol table
 - ▶ Call `symbolTable.enterScope()`
- ▶ New identifiers detected
 - ▶ Classes, methods, fields, parameters, variables
 - ▶ Create a new Record with id and type
 - ▶ Add the Record to the current scope
 - ▶ `symbolTable.put(id, new Record(id, type))`
 - ▶ Leaving a scope
 - ▶ `symbolTable.exitScope()`
- ▶ Focus on identifier declarations; their use and definitions is for later
- ▶ Single AST traversal is enough
 - ▶ Implement a tree traversal
- ▶ When do we shift scope?
- ▶ In which AST nodes do we put new records in the table?

Generating the ST - example



```

1 Node() {
2   ...
3   execute(...) {...}
4   ...
5 }
6 MethodDeclaration :: Node() {
7   ...
8   execute(...) {
9     retType = execute(child[0]); //return type
10    name = execute(child[1]); //method name
11    currentMethod = new Method(mName, retType);
12    symbolTable.put(mName, currentMethod); //add to ST
13    currentClass.addMethod(mName, currentMethod); //add to current class
14
15    symbolTable.enterScope(); //enter method scope
16    execute(child[2]); //visit parameterList
17    execute(child[3]); //visit methodBody
18    symbolTable.exitScope(); //exit scope
19    ...
20  }
21  ...
22 }
  
```

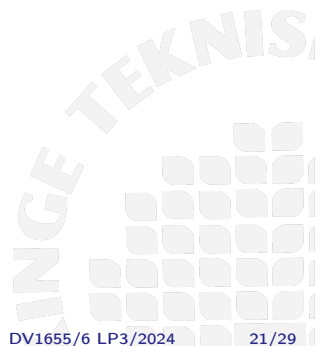
Outline

Overview

Tree Traversal

Symbol table

Semantic Analysis



Semantic Analysis

- ▶ A separate phase of the compiler
 - ▶ Input: AST and symbol table
 - ▶ Implemented as a new tree traversal (may require more than one tree traversal)
 - ▶ Requires evaluation of types of subtrees (in post-order)
- ▶ Tasks
 - ▶ Verify that all identifiers are declared
 - ▶ Type checking
 - ▶ Expressions: e.g., In `a+b`; both `a` and `b` should be integers
 - ▶ Statements: e.g., In `a = b`; type of `a` and `b` should be the same
 - ▶ Methods: In `int a(int c) ... return b`; the type of return expression (`b`) should be the same as type of the method declaration (`int`).
 - ▶ Method calls: In `a(x)`; the type of `x` and `c` should be the same
 - ▶ Array access: In `b[a]`; `b.length` the type of `a` must be integer; the type of `b` must be `int[]`;
- ▶ Out of scope
 - ▶ Inheritance, polymorphic calls, and other OO features

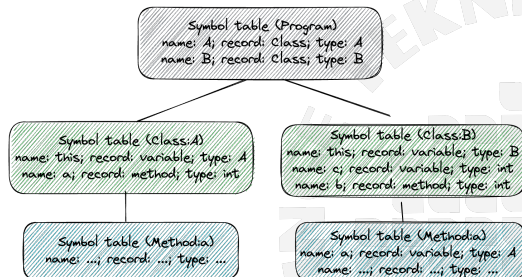
Undeclared identifiers

- ▶ All declared identifiers are stored in the symbol table
 - ▶ Lookup: `symbolTable.lookup(c)` should not be null
- ▶ Dealing with method calls
 - ▶ Method `a()` is not visible from `b()`
 - ▶ Simple lookup for `a()` will return null
 - ▶ Solution
 - ▶ `Lookup(a).getType()` will result with `A`;
 - ▶ `Lookup(A).getMethod(a)` will not be null;

```

1  class A {
2      int a(){...}
3  }
4
5  class B{
6      int c;
7      int b(){
8          A a;
9          ...
10         res = a.a() + c;
11     }
12 }

```



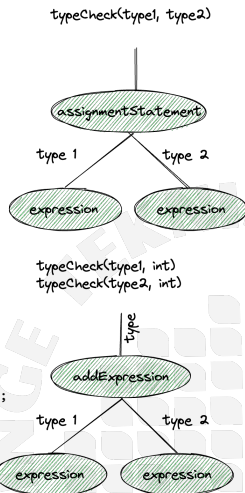
Type checking

- ▶ Type information is transported upwards
 - ▶ The evaluate method returns the type of a child
 - ▶ Leaf nodes return their types
 - ▶ Other nodes need to evaluate the type of their children
- ▶ When the type of all children nodes are evaluated, we perform type checking

```

1 Node() {
2   ...
3   semanticAnalysis(...) {...}
4   ...
5 }
6 AssignmentStatement :: Node() {
7   ...
8   semanticAnalysis(...) {
9     ...
10    lhsType = evaluate(child[0]);
11    rhsType = evaluate(child[1]);
12    if(lhsType != rhsType)
13      print("Type mismatch");
14    ...
15  }
16 }
17 ...
18 AddExpression :: Node() {
19   ...
20   semanticAnalysis(...) {
21     ...
22     for(Node i : children)
23       rhsType = evaluate(children[i]);
24     if(lhsType != "int")
25       print("Type mismatch");
26     ...
27   }
28 }

```



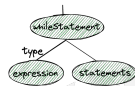
If/While Statements

- ▶ We need to evaluate the type of the boolean expression
- ▶ The statements AST nodes do not have any types
- ▶ Other AST nodes (e.g., expressions) have types

```

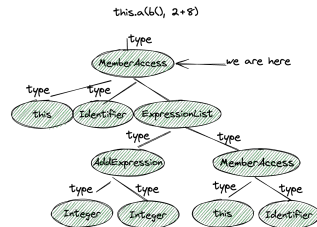
1 Node() {
2     ...
3     semanticAnalysis(...) {...}
4     ...
5 }
6 WhileStatement :: Node() {
7     ...
8     semanticAnalysis(...) {
9         ...
10        conditionType = evaluate(child[0]);
11        if(conditionType != "boolean")
12            print("Type mismatch");
13        ...
14    }
15    ...
16 IfExpression :: Node() {
17    ...
18    semanticAnalysis(...) {
19        ...
20        conditionType = evaluate(child[0]);
21        if(conditionType != "boolean")
22            print("Type mismatch");
23        ...
24    }
25 }

```



Method Calls

- ▶ Get the target (this) type class
 - ▶ Lookup if class exists in ST
- ▶ Get method name
 - ▶ Lookup method in the scope of the target class
 - ▶ Check if method return type is same as declared type
- ▶ Evaluate all parameters
 - ▶ Check if the type of parameters corresponds to the declared method parameters

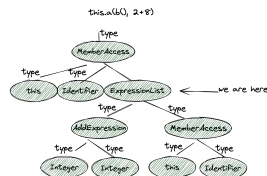


```

1 Node() {
2   ...
3   semanticAnalysis(...) {...}
4   ...
5 }
6 MethodCall :: Node() {
7   ...
8   semanticAnalysis(...) {
9     ...
10    targetType = evaluate(child[0]);
11    class = symbolTable.lookup(targetType);
12    methodName = evaluate(child[1]);
13    Method = class.getMethod(methodName);
14    ...
15    // evaluate parameter lists
16    ...
17  }
18 }
  
```

Parameter List

- ▶ Check the size of defined list of parameters and children
- ▶ For each child
 - ▶ Check if type is the same as the defined parameter type
- ▶ You need a reference to the current method



```

1 Node() {
2     ...
3     semanticAnalysis(...) {...}
4     ...
5 }
6 ExpressionList :: Node() {
7     ...
8     semanticAnalysis(...) {
9         ...
10        paramList = currentMethod.children;
11        if(paramList.size() != children.size())
12            print("parameter list size mismatch");
13        for(int i = 0; i < children.size(); i++) {
14            type = evaluate(children[i]);
15            definedType = evaluate(paramList/children[i]);
16            if(type != definedType)
17                print("parameter type mismatch");
18        }
19        ...
20    }
21 }

```

Assignment 2: Semantic Analysis

- ▶ **Part 1:** Symbol Table construction
 - ▶ AST + ST is input to the code generation
- ▶ **Part 2:** Semantic Analysis
 - ▶ Type analysis: expressions, statements, method declaration, method calls, array access, length member for arrays, ...
 - ▶ Check for undeclared identifiers, check for duplicate identifiers, ...

References

- ▶ *"Compilers: Principles, Techniques, and Tools"* second edition, by Aho et al,
 - ▶ **Chapter 6.5** - Type Checking